Validation Report (Section 11)

WM-8 five-stage pipelined washing-machine microcontroller – ELL7282/ELL782 Introduction to Computer Architecture – Assignment 2

1 Method and reproducibility

- **Device under test.** The six test benches `tests/t1–t6_*.circ` each contain *the same* sub-circuits as `washing_pipeline.cir` (`cpu`, `mmio`, `timer8`, `safety` and all gate-level blocks). Only the top level differs. Its pieces are:
  - the ROM holds the test program;
  - a power-on-reset flip-flop asserts `RESET` for cycle 0;
  - a 16-bit cycle counter drives a *stimulus ROM* that supplies `START`, `DOOR_CLOSED` and `MODE` (the sensor values are listed below);
  - a small plant model supplies `WATER_LEVEL` and `DRUM_RPM`: water +3 every 16 cycles while `VALVE_IN`=1 and −7 (saturating at 0) every 16 cycles while `PUMP_OUT`=1; drum 200 rpm while `MOTOR_EN_ACTUAL`=1, otherwise −40 per cycle down to 0;
  - `TEMP` is the constant `0x19`;
  - an output pin `halt` stops the run after a fixed number of cycles.

  Inputs change only right after a rising edge, so they are stable at every sampling edge.
- **Observation.** One row per clock cycle with 36 monitored signals: pipeline state, stall/flush, forwarding, write-back and store ports, `RAM_WE`, actuator latch, display, gated motor output, `TRIP`, timer registers and sensor values. The *observed* column below was extracted automatically by `tools/make_reports.py` from these rows. The rows were recorded by cycle-accurate simulation of the test-bench `.circ` netlists with `tools/circsim.py`. That simulator rebuilds connectivity from the XML geometry and also checks for width conflicts, multiple drivers, undriven nets and combinational loops (none found).
- **Reproducing in Logisim.** `java -jar logisim-generic-2.7.1.jar tests/t1_hazards.circ -tty table > t1.txt`, then `diff t1.txt tests/expected/t1_hazards.txt`; `tests/run_logisim_tests.sh` does this for all six. The same files can also be opened in the GUI and clocked by hand (`RUNNING_IN_LOGISIM.md`).
- **Cross-checks.** For t1–t3 the ordered list of register write-backs and stores is compared with an instruction-level reference model (`tools/isa_ref.py`). In a strict mode in which every register and RAM byte starts *undefined*, the reference model also confirms that `program.asm` never reads uninitialised state (normal path and safety-trip path).

Result: 39 of 39 checks passed .

Sensor inputs used

Bench	Inputs (cycle numbers are rising-edge counts; values hold until the next change)
t1–t4	`DOOR_CLOSED`=1, `START`=0, `MODE`=00, `WATER_LEVEL`=0, `TEMP`=`0x19`, `DRUM_RPM`=0 (not used by the programs)
t5	`DOOR`: 1, 0 @40, 1 @52, 0 @88, 1 @108; `START`: 0, 1 @64, 0 @76, 1 @92, 0 @100, 1 @120, 0 @140
t6	`DOOR`=1; `START`: 1 @32, 0 @128, 1 @1600, 0 @1696; `MODE`: 00, 01 @480 (during run 1), 01 @1600, 00 @2400 (during run 2); `WATER_LEVEL`/`DRUM_RPM` from the plant model (fill reads . . . 96, 99, 102 and . . . 156, 159, 162)

2 Group 1 – ALU forwarding, load-use stall, youngest-writer priority (`t1_hazards`)

Initial state: all pipeline entries invalid after reset. Registers are initialised by the program (R1=01, R2=10, R3=03, R4=20, RAM[10]=03). Cycle traces are in `pipeline_traces.pdf`.

ID	Initial inputs / state, stimulus	Expected result	Observed result	P/F
1a	§8.1 `ADD R1,R2; SUB R3,R1`; R1=01, R2=10, R3=03	no stall; SUB takes R1 from EX/MEM; R3=F2	cyc 10: stall=0, FwdB=EX/MEM; R3=F2 (cyc 12)	**PASS**
1b	§8.2 `LD R1,[R2]; ADD R3,R1`; RAM[10]=03	exactly 1 stall cycle; PC and IF/ID hold; bubble (V=0) in EX; R1 from MEM/WB; R3=F5	stall cycles [11]; cyc 12: IDEX.V=0, PC held at 0A; cyc 13: FwdB=MEM/WB; R3=F5	**PASS**
1c	§8.3 `ADD R1,R2; SUB R1,R3; ST R1,[R4]`	store data from youngest writer (SUB, EX/MEM) = 1E, not older ADD (MEM/WB) = 13	cyc 16: FwdA=EX/MEM while MEM/WB holds pc 0A; RAM[20]←1E (cyc 17)	**PASS**
1d	read-back `LD R5,[R4]; ST R5,[DISPLAY]; HALT; LDI R6,FF`	DISPLAY=1E; HALT stops machine; younger LDI R6 never writes	DISPLAY=1E; HALTED=1; writes to R6: 0	**PASS**

ID	Initial inputs / state, stimulus	Expected result	Observed result	P/F
1e	whole program vs. ISA reference model	identical ordered list of register/memory writes	15 writes, identical	**PASS**

3 Group 2 – taken branch kills a wrong-path actuator store and a wrong-path HALT (`t2_branch_halt`)

Initial state: reset. R2=`0x21` (VALVE+LOCK pattern), R4=`0xF4` (actuator latch), R3=07, R1=09.

ID	Initial inputs / state, stimulus	Expected result	Observed result	P/F
2a	§8.4 `SUB R1,R1; BEQZ R1,T; ST R2,[Rvalve]; ADD R3,R4`; R2=21, Rvalve=F4	branch taken in EX (R1 via EX/MEM); IF/ID and ID/EX invalidated; wrong-path store reaches MEM with V=0; VALVE/ACT never written	cyc 9: flush=1, FwdA=EX/MEM; cyc 10: IFID.V=0, IDEX.V=0; cyc 11: ST in MEM V=0; valid stores to F4: 0; ACT=00 throughout	**PASS**
2b	wrong-path `ADD R3,R4`	R3 keeps 07 (DISPLAY shows 07)	writes to R3: 07 (only the LDI); DISPLAY=07 at cyc 18	**PASS**
2c	`BNEZ R1,bad` with R1=0	not taken, no flush	cyc 12: flush=0	**PASS**
2d	wrong-path `HALT` at 0B (after taken BNEZ) and 0F/10 (after JMP)	flushed before EX; execution continues to DISPLAY=AA	wrong-path HALTs valid in EX: 0; DISPLAY=AA at cyc 23; HALTED first at cyc 23 by HALT @13	**PASS**
2e	real `HALT` followed by `ST R2,[R4]`	younger store killed; fetch stops; remains halted	store @14 valid in MEM: False; ACT=00; PC frozen at 15	**PASS**
2f	whole program vs. ISA reference model	identical write sequence	9 writes, identical	**PASS**

4 Group 3 – RAM store/load and reserved-address isolation (`t3_ram_reserved`)

Initial state: reset. The program first writes RAM[00]=5A, RAM[6F]=33 and RAM[7F]=A5, then stores `0x77` to addresses that would alias these locations if bit 7 were ignored (`0x80`→00, `0xEF`→6F, `0xFF`→7F) and to read-only `0xF0`.

ID	Initial inputs / state, stimulus	Expected result	Observed result	P/F
3a	`ST` to RAM 00, 6F, 7F	RAM write-enable exactly 3 times	RAM_WE at cyc 6 [00]←5A, cyc 9 [6F]←33, cyc 12 [7F]←A5	**PASS**
3b	`ST 77` to 80, EF (reserved), FF (unimpl.), F0 (read-only)	no RAM write (no aliasing), no latch change	4 stores seen on the bus, RAM_WE during them: 0; ACT/DISPLAY unchanged	**PASS**
3c	loads from 7F, 00, 6F after the reserved stores	R6=A5, R7=5A, R1=33 (unchanged)	R6=A5, R7=5A, R1=33	**PASS**
3d	loads from 80, EF (reserved) and F7 (unimplemented)	read as 00	R0=00, R3=00, R5=00	**PASS**
3e	whole program vs. ISA reference model	identical write sequence	31 writes, identical	**PASS**

5 Group 4 – timer (`t4_timer`)

Initial state: reset (`COUNT`=`LIMIT`=0, `EN`=`MATCH`=0).

ID	Initial inputs / state, stimulus	Expected result	Observed result	P/F
4a	LIMIT=6, CTRL=03 (clear+enable)	COUNT=0 after the CTRL edge, +1 per clock; MATCH set on the edge where COUNT becomes 6	CTRL write cyc 11; COUNT=00 at cyc 12; MATCH=1 at cyc 18 with COUNT=06	**PASS**
4b	polling loops `LD R2,[T_STATUS]; BEQZ R2` (load-use stall)	COUNT increments during every stall cycle	stall cycles [16, 21, 31, 36, 41, 53]: COUNT +1 each	**PASS**

ID	Initial inputs / state, stimulus	Expected result	Observed result	P/F
4c	keep running after MATCH	MATCH stays 1 (sticky) while COUNT passes LIMIT	MATCH=1 from cyc 18 to 30, COUNT reached 12	**PASS**
4d	re-arm: LIMIT=0A, CTRL=03	clear wins: COUNT=0, MATCH=0 on the CTRL edge; MATCH again at COUNT=0A	cyc 31: COUNT=00, MATCH=0; MATCH at cyc 41 (COUNT=0A)	**PASS**
4e	timer stopped (CTRL=00), then LIMIT := current COUNT	no retroactive MATCH	cyc 57: COUNT=05, LIMIT=05, MATCH=0; DISPLAY(MATCH)=00	**PASS**
4f	LIMIT=03, CTRL=01 (enable, no clear), then `HALT`	timer keeps ticking while halted; COUNT wraps FF→00; MATCH at COUNT=03	wrap at cyc 315 (HALTED=1); MATCH at cyc 318 (COUNT=03)	**PASS**

Cyc	STALL	HALTED	EN	COUNT	LIMIT	MATCH
10	0	0	0	00	06	0
11	1	0	0	00	06	0
12	0	0	1	00	06	0
13	0	0	1	01	06	0
14	0	0	1	02	06	0
15	0	0	1	03	06	0
16	1	0	1	04	06	0
17	0	0	1	05	06	0
18	0	0	1	06	06	1
19	0	0	1	07	06	1
			...			
313	0	1	1	FE	03	0
314	0	1	1	FF	03	0
315	0	1	1	00	03	0
316	0	1	1	01	03	0
317	0	1	1	02	03	0
318	0	1	1	03	03	1
319	0	1	1	04	03	1

Excerpt: interval 1 (stall cycles marked by `STALL`=1, the count still advances) and the wrap-around while halted.

6 Group 5 – safety interlock (`t5_safety`)

Initial state: reset (`TRIP`=0, `START_PREV`=0). The program requests `MOTOR_EN` and keeps the request until it sees a START press and release, then clears the request and waits for `TRIP`=0 before requesting the motor again. The physical output `MOTOR_EN_ACTUAL` is checked separately from the CPU request (`ACT` bit 2).

ID	Initial inputs / state, stimulus	Expected result	Observed result	P/F
5a	DOOR=1, software writes ACT=24 (LOCK+MOTOR_EN)	ActualMotorEnable=1	cyc 11: ACT=24, actual=1	**PASS**
5b	open door at cyc 40 (request still 1)	actual motor drops in the same cycle (combinational); TRIP latched at next edge	cyc 40: MOTOR_EN req=1, actual=0, TRIP=0; cyc 41: TRIP=1	**PASS**
5c	re-close door at cyc 52, request still 1	no automatic restart	cyc 52-61: actual=0, TRIP=1, request=1	**PASS**
5d	START 0→1 at cyc 64 while MOTOR_EN request still active	not a valid re-arm: TRIP stays 1	cyc 65: TRIP=1	**PASS**
5e	software clears request (cyc 83); START edge with door open (cyc 92)	not a valid re-arm: TRIP stays 1	cyc 93: TRIP=1	**PASS**
5f	door closed, request off, new START edge at cyc 120	TRIP cleared; motor runs only after software requests it again	cyc 121: TRIP=0; actual=1 again at cyc 127 (ACT=24); DISPLAY=03	**PASS**

Cyc	DOOR	START	ACT	MOTOR_EN req	Actual	TRIP	note
0	1	0	00	0	0	0	
11	1	0	24	1	1	0	
40	0	0	24	1	0	0	
41	0	0	24	1	0	1	
52	1	0	24	1	0	1	
64	1	1	24	1	0	1	
76	1	0	24	1	0	1	
83	1	0	20	0	0	1	
88	0	0	20	0	0	1	
92	0	1	20	0	0	1	
100	0	0	20	0	0	1	
108	1	0	20	0	0	1	
120	1	1	20	0	0	1	
121	1	1	20	0	0	0	
127	1	1	24	1	1	0	
140	1	0	24	1	1	0	

Every change of `DOOR`, `START`, `ACT`, actual motor enable or `TRIP`.

7 Group 6 – end-to-end wash, two MODE settings (`t6_wash_e2e`, `program.asm`)

Initial state: reset, `DOOR_CLOSED`=1, empty drum. Run 1 uses `MODE`=00 (NORMAL: target 100, wash 64, spin 32, drain ≤10). Run 2 starts after run 1 has completed and the buzzer is on, with `MODE`=01 (HEAVY: 160, 96, 48, ≤10). `MODE` is changed during both runs to show that the snapshot is used. The overshoot samples are 99→102 (target 100) and 159→162 (target 160). The drain thresholds are also crossed by a jump (11→4 and 15→8).

ID	Initial inputs / state, stimulus	Expected result	Observed result	P/F
6a	NORMAL (MODE=00): START=1, DOOR=1; plant +3 per 16 cycles	fill until WATER_LEVEL ≥ 100 (unsigned) although reading jumps past 100	last samples 99, 99, 99, 102; valve closed at cyc 619 with WATER_LEVEL=102	**PASS**
6b	wash phase	LIMIT=64; motor direction alternates; motor off at MATCH	LIMIT=64; DIR changed 3 times; motor off at cyc 710 (COUNT=75, MATCH=1)	**PASS**
6c	drain phase (plant −7 per 16 cycles)	pump until WATER_LEVEL ≤ 10 (unsigned)	samples . . . 11, 11, 4; spin starts cyc 950	**PASS**
6d	spin phase	timer cleared/re-armed, LIMIT=32, MATCH at 32	LIMIT=32, COUNT=2 at spin start, MATCH at cyc 980 (COUNT=32)	**PASS**
6e	completion	all actuators off; unlock only after drum RPM=0; BUZZER=1; DISPLAY=08	motor/pump off cyc 989; RPM=0 cyc 994; ACT=40 (unlock+buzzer) cyc 1004; DISPLAY=08	**PASS**
6g	HEAVY (MODE=01): START=1, DOOR=1; plant +3 per 16 cycles	fill until WATER_LEVEL ≥ 160 (unsigned) although reading jumps past 160	last samples 159, 159, 159, 162; valve closed at cyc 2507 with WATER_LEVEL=162	**PASS**
6h	wash phase	LIMIT=96; motor direction alternates; motor off at MATCH	LIMIT=96; DIR changed 5 times; motor off at cyc 2626 (COUNT=103, MATCH=1)	**PASS**
6i	drain phase (plant −7 per 16 cycles)	pump until WATER_LEVEL ≤ 10 (unsigned)	samples . . . 15, 15, 8; spin starts cyc 2998	**PASS**
6j	spin phase	timer cleared/re-armed, LIMIT=48, MATCH at 48	LIMIT=48, COUNT=2 at spin start, MATCH at cyc 3044 (COUNT=48)	**PASS**
6k	completion	all actuators off; unlock only after drum RPM=0; BUZZER=1; DISPLAY=18	motor/pump off cyc 3055; RPM=0 cyc 3060; ACT=40 (unlock+buzzer) cyc 3070; DISPLAY=18	**PASS**
6m	MODE changed during run 1 (cyc 480) and run 2 (cyc 2400)	parameters of the running cycle unchanged	run 1 still used target 100 / LIMIT 64 / 32; run 2 still used 160 / 96 / 48 (rows 6a–6k)	**PASS**

Event	NORMAL	HEAVY
DOOR_LOCK commanded (START accepted)	67	1631
VALVE_IN on	78	1642
VALVE_IN off	619	2507
wash motor off (MATCH)	710	2626
spin start	950	2998
spin MATCH	980	3044
motor/pump off	989	3055
drum stopped (RPM=0)	994	3060
unlock + BUZZER	1004	3070
water level at valve close	102	162
last drain sample	4	8

Phase timeline (cycle numbers) of both runs. DISPLAY sequence NORMAL 01, 02, 03, 04, 05, 06, 07, 08; HEAVY 12, 13, 14, 15, 16, 17, 18.

A Remaining test program listings

t3_ram_reserved.asm

Addr	Hex	Fmt	Label	Assembly	Comment
00	125A	I		LDI R1, 0x5A	
01	1400	I		LDI R2, 0x00	
02	7280	R		ST R1, [R2]	RAM[0x00] = 0x5A
03	1633	I		LDI R3, 0x33	
04	146F	I		LDI R2, 0x6F	
05	7680	R		ST R3, [R2]	RAM[0x6F] = 0x33
06	16A5	I		LDI R3, 0xA5	
07	147F	I		LDI R2, 0x7F	
08	7680	R		ST R3, [R2]	RAM[0x7F] = 0xA5
09	1A77	I		LDI R5, 0x77	
0A	1880	I		LDI R4, 0x80	
0B	7B00	R		ST R5, [R4]	reserved 0x80: ignored (would alias RAM[0x00])
0C	18EF	I		LDI R4, 0xEF	
0D	7B00	R		ST R5, [R4]	reserved 0xEF: ignored (would alias RAM[0x6F])
0E	18FF	I		LDI R4, 0xFF	
0F	7B00	R		ST R5, [R4]	unimplemented peripheral: ignored (alias RAM[0x7F])
10	18F0	I		LDI R4, 0xF0	
11	7B00	R		ST R5, [R4]	read-only IN_STATUS: ignored
12	6C80	R		LD R6, [R2]	R6 = RAM[0x7F] = 0xA5
13	1400	I		LDI R2, 0x00	
14	6E80	R		LD R7, [R2]	R7 = RAM[0x00] = 0x5A
15	146F	I		LDI R2, 0x6F	
16	6280	R		LD R1, [R2]	R1 = RAM[0x6F] = 0x33
17	1880	I		LDI R4, 0x80	
18	6100	R		LD R0, [R4]	R0 = 0x00 (reserved reads 0)
19	18EF	I		LDI R4, 0xEF	
1A	6700	R		LD R3, [R4]	R3 = 0x00
1B	18F7	I		LDI R4, 0xF7	
1C	6B00	R		LD R5, [R4]	R5 = 0x00 (unimplemented peripheral)
1D	18F5	I		LDI R4, 0xF5	
1E	7D00	R		ST R6, [R4]	DISPLAY = 0xA5
1F	F000	N		HALT	

t4_timer.asm

Addr	Hex	Fmt	Label	Assembly	Comment
00	1EF9	I		LDI R7, 0xF9	T_LIMIT
01	1CFA	I		LDI R6, 0xFA	T_CTRL
02	1AFB	I		LDI R5, 0xFB	T_STATUS
03	18F8	I		LDI R4, 0xF8	T_COUNT
04	1206	I		LDI R1, 6	
05	73C0	R		ST R1, [R7]	LIMIT = 6
06	1203	I		LDI R1, 0x03	
07	7380	R		ST R1, [R6]	interval 1: clear + enable
08	6540	R	poll1	LD R2, [R5]	
09	9408	I		BEQZ R2, poll1	load-use stall every iteration (timer keeps counting)
0A	6700	R		LD R3, [R4]	COUNT kept counting past LIMIT
0B	6540	R		LD R2, [R5]	MATCH still 1 (sticky)
0C	120A	I		LDI R1, 10	
0D	73C0	R		ST R1, [R7]	LIMIT = 10
0E	1203	I		LDI R1, 0x03	
0F	7380	R		ST R1, [R6]	interval 2: clear COUNT+MATCH, re-arm
10	6540	R		LD R2, [R5]	MATCH = 0 after clear
11	6540	R	poll2	LD R2, [R5]	
12	9411	I		BEQZ R2, poll2	
13	1200	I		LDI R1, 0x00	
14	73C0	R		ST R1, [R7]	LIMIT = 0 (comparison disabled)
15	1203	I		LDI R1, 0x03	
16	7380	R		ST R1, [R6]	clear + enable
17	0000	N		NOP	
18	0000	N		NOP	
19	0000	N		NOP	
1A	1200	I		LDI R1, 0x00	

Addr	Hex	Fmt	Label	Assembly	Comment
1B	7380	R		ST R1, [R6]	disable (no clear): COUNT frozen
1C	6700	R		LD R3, [R4]	R3 = frozen COUNT
1D	77C0	R		ST R3, [R7]	LIMIT = COUNT -> MATCH must stay 0
1E	6540	R		LD R2, [R5]	R2 = 0
1F	12F5	I		LDI R1, 0xF5	
20	7440	R		ST R2, [R1]	DISPLAY = MATCH = 0
21	1203	I		LDI R1, 0x03	
22	73C0	R		ST R1, [R7]	LIMIT = 3
23	1201	I		LDI R1, 0x01	
24	7380	R		ST R1, [R6]	enable, no clear: COUNT runs ..FF -> 00 -> 03 => MATCH
25	F000	N		HALT	timer keeps ticking while halted

t5_safety.asm

Addr	Hex	Fmt	Label	Assembly	Comment
00	1EF4	I		LDI R7, 0xF4	ACT
01	1CF0	I		LDI R6, 0xF0	IN_STATUS
02	1AF5	I		LDI R5, 0xF5	DISPLAY
03	1801	I		LDI R4, 0x01	START mask
04	1610	I		LDI R3, 0x10	TRIP mask
05	1224	I		LDI R1, 0x24	
06	73C0	R		ST R1, [R7]	request DOOR_LOCK + MOTOR_EN
07	1201	I		LDI R1, 0x01	
08	7340	R		ST R1, [R5]	DISPLAY = 1 : motor requested
09	6580	R	waitS	LD R2, [R6]	
0A	5500	R		AND R2, R4	
0B	9409	I		BEQZ R2, waitS	wait for a START press (request still active)
0C	6580	R	waitR	LD R2, [R6]	
0D	5500	R		AND R2, R4	
0E	A40C	I		BNEZ R2, waitR	wait for START release
0F	1220	I		LDI R1, 0x20	
10	73C0	R		ST R1, [R7]	software clears MOTOR_EN request
11	1202	I		LDI R1, 0x02	
12	7340	R		ST R1, [R5]	DISPLAY = 2 : waiting for re-arm
13	6580	R	waitT	LD R2, [R6]	
14	54C0	R		AND R2, R3	
15	A413	I		BNEZ R2, waitT	until hardware TRIP is cleared
16	1224	I		LDI R1, 0x24	
17	73C0	R		ST R1, [R7]	request motor again
18	1203	I		LDI R1, 0x03	
19	7340	R		ST R1, [R5]	DISPLAY = 3 : running again
1A	F000	N		HALT	